BÁO CÁO THỰC HÀNH BUỔI 7

BẢO MẬT TRONG ỨNG DỤNG QUẢN LÝ LỊCH HỌP

Môn học: ASP.NET Core MVC | Người thực hiện: ..... | Ngày: 28/10/2026

Mục tiêu và phạm vi

Báo cáo trình bày việc áp dụng yêu cầu thực hành về Cookie, Session và ASP.NET Core Identity vào hệ thống quản lý lịch họp MeetingScheduler. Các cơ chế được tích hợp với tài khoản, lịch họp và vai trò có sẵn của ứng dụng; giao diện chỉ hiển thị thông tin hữu ích cho người dùng.

Yêu cầu thực hành	Cách áp dụng trong hệ thống	Minh chứng
Persistent Cookie	Ghi nhớ lượt truy cập, thời điểm gần nhất và chức năng thường dùng trên thiết bị trong 30 ngày.	UserActivityStateService.cs
Session	Giữ thông tin phiên làm việc; khởi tạo khi đăng nhập, hết hạn sau 30 phút không hoạt động và xóa khi đăng xuất.	Program.cs, AccountController.cs
ASP.NET Core Identity	Dùng UserManager và kho người dùng tùy chỉnh để kiểm tra/đổi mật khẩu trên bảng User hiện tại.	Program.cs, MeetingUserStore.cs, AccountService.cs
Authentication / Authorization	Dùng cookie xác thực, kiểm tra trạng thái tài khoản, phân quyền theo vai trò và quyền sở hữu cuộc họp.	Program.cs, MeetingController.cs

Cấu trúc bảo mật

Cookie đăng nhập được ASP.NET Core bảo vệ và mang Claims đã ký. Cookie ghi nhớ hoạt động chỉ chứa số liệu sử dụng, không chứa mật khẩu hoặc quyền. Session lưu trạng thái làm việc phía máy chủ; quyết định cho phép thao tác vẫn dựa trên Claims và dữ liệu tài khoản trong cơ sở dữ liệu.

1. Ghi nhớ hoạt động bằng Cookie

Sau mỗi yêu cầu GET của người dùng đã đăng nhập, middleware ghi tăng số lượt truy cập, thời điểm truy cập và bộ đếm theo chức năng như xem lịch, xem chi tiết hoặc theo dõi RSVP. Danh sách chức năng được giới hạn ở tám mục để giữ dữ liệu nhỏ.

Giá trị được mã hóa/bảo vệ bằng ASP.NET Core Data Protection trước khi gửi về trình duyệt. Cookie hết hạn sau 30 ngày, đặt HttpOnly và SameSite=Lax; cờ Secure bật khi yêu cầu đi qua HTTPS. Cookie này chỉ phục vụ thống kê sử dụng, không được dùng để xác thực hoặc phân quyền.

Services/UserActivityStateService.cs — SOURCE LINES 36-58

```
36        public void RecordRequest(HttpContext context, string feature)
37        {
38            var usage = ReadCookie(context);
39            usage.VisitCount++;
40            usage.LastAccessUtc = DateTimeOffset.UtcNow;
41            usage.ExpiresUtc = DateTimeOffset.UtcNow.Add(CookieLifetime);
42            usage.FeatureHits[feature] = usage.FeatureHits.GetValueOrDefault(feature) + 1;
43            usage.FeatureHits = usage.FeatureHits.OrderByDescending(x => x.Value).Take(8)
44                .ToDictionary(x => x.Key, x => x.Value, StringComparer.Ordinal);
45            context.Items[CookieName] = usage;
46
47            context.Response.Cookies.Append(CookieName, protector.Protect(JsonSerializer.Serialize(usage)), new
      CookieOptions
48            {
49                Expires = usage.ExpiresUtc,
50                HttpOnly = true,
51                Secure = context.Request.IsHttps,
52                SameSite = SameSiteMode.Lax,
53                IsEssential = true
54            });
55            context.Session.SetInt32(RequestCountKey, (context.Session.GetInt32(RequestCountKey) ?? 0) + 1);
56            context.Session.SetString(LastFeatureKey, feature);
57        }
58
```

Hình 1. Ghi nhận hoạt động và tạo Cookie được bảo vệ trong UserActivityStateService.cs

Các thuộc tính an toàn

Thuộc tính	Mục đích trong ứng dụng
Expires	Lưu thống kê qua các lần mở trình duyệt trong thời hạn 30 ngày.
HttpOnly	Ngăn JavaScript phía trình duyệt đọc trực tiếp giá trị Cookie.
Secure	Chỉ gửi Cookie qua HTTPS khi ứng dụng đang sử dụng HTTPS.
SameSite=Lax	Giảm khả năng trình duyệt gửi Cookie trong yêu cầu liên trang không mong muốn.
Data Protection	Bảo vệ tính bí mật và toàn vẹn dữ liệu thống kê được lưu trong Cookie.

2. Quản lý phiên làm việc

Ứng dụng bật Session với thời gian chờ 30 phút, Cookie phiên HttpOnly, SameSite=Lax và SecurePolicy theo giao thức yêu cầu. Khi đăng nhập thành công, hệ thống tạo trạng thái phiên gồm mã người dùng, vai trò hiển thị, thời điểm đăng nhập và bộ đếm hoạt động. Khi đăng xuất, Session được xóa.

Program.cs — SOURCE LINES 23-31

```
23  builder.Services.AddSession(options =>
24  {
25      options.IdleTimeout = TimeSpan.FromMinutes(30);
26      options.Cookie.Name = ".MeetingScheduler.Session";
27      options.Cookie.HttpOnly = true;
28      options.Cookie.SameSite = SameSiteMode.Lax;
29      options.Cookie.SecurePolicy = CookieSecurePolicy.SameAsRequest;
30      options.Cookie.IsEssential = true;
31  });
```

Hình 2. Cấu hình Session trong Program.cs

Controllers/AccountController.cs — SOURCE LINES 32-41

```
32              var identity = new ClaimsIdentity(new[]
33              {
34                  new Claim(ClaimTypes.NameIdentifier,
        account.UserId.ToString(System.Globalization.CultureInfo.InvariantCulture)),
35                  new Claim(ClaimTypes.Name, account.FullName),
36                  new Claim(ClaimTypes.Email, account.Email),
37                  new Claim(ClaimTypes.Role, account.RoleName)
38              }, CookieAuthenticationDefaults.AuthenticationScheme);
39              await HttpContext.SignInAsync(CookieAuthenticationDefaults.AuthenticationScheme,
40                  new ClaimsPrincipal(identity), new AuthenticationProperties { IsPersistent = model.RememberMe });
41              activity.StartSession(HttpContext, account.UserId, account.RoleName);
```

Hình 3. Đăng nhập, tạo Claims và khởi tạo trạng thái phiên trong AccountController.cs

Controllers/AccountController.cs — SOURCE LINES 48-52

```
48          public async Task<IActionResult> Logout()
49          {
50              activity.ClearSession(HttpContext);
51              await HttpContext.SignOutAsync(CookieAuthenticationDefaults.AuthenticationScheme);
52              return RedirectToAction(nameof(Login));
```

Hình 4. Xóa Session và kết thúc phiên xác thực khi đăng xuất

Giá trị vai trò trong Session chỉ hỗ trợ lưu trạng thái làm việc. Bộ lọc quyền không dựa vào giá trị đó; quyền truy cập dùng Claims trong cookie xác thực và được kiểm tra lại với cơ sở dữ liệu.

3. Tích hợp ASP.NET Core Identity

Identity Core được đăng ký với lớp User hiện có. Chính sách mật khẩu yêu cầu tối thiểu tám ký tự, có chữ hoa, chữ thường, chữ số và ký tự đặc biệt. Email phải duy nhất.

Program.cs — SOURCE LINES 32-40

```
builder.Services.AddIdentityCore<User>(options =>
{
    options.Password.RequiredLength = 8;
    options.Password.RequireDigit = true;
    options.Password.RequireLowercase = true;
    options.Password.RequireUppercase = true;
    options.Password.RequireNonAlphanumeric = true;
    options.User.RequireUniqueEmail = true;
}).AddUserStore<MeetingUserStore>();
```

Hình 5. Đăng ký Identity Core và chính sách mật khẩu trong Program.cs

Bảng User của dự án không kế thừa IdentityUser và đang được các chức năng lịch họp sử dụng. Vì vậy MeetingUserStore triển khai IUserPasswordStore và IUserEmailStore để nối UserManager với bảng hiện có, không tạo thêm bộ bảng Identity hoặc thay đổi lược đồ dữ liệu.

Services/MeetingUserStore.cs — SOURCE LINES 47-55

```
    public Task SetPasswordHashAsync(User user, string? passwordHash, CancellationToken cancellationToken)
    { user.PasswordHash = passwordHash ?? ""; return Task.CompletedTask; }
    public Task<string?> GetPasswordHashAsync(User user, CancellationToken cancellationToken) =>
    Task.FromResult<string?>(user.PasswordHash);
    public Task<bool> HasPasswordAsync(User user, CancellationToken cancellationToken) =>
    Task.FromResult(!string.IsNullOrWhiteSpace(user.PasswordHash));

    public Task SetEmailAsync(User user, string? email, CancellationToken cancellationToken) { user.Email =
    Normalize(email); return Task.CompletedTask; }
    public Task<string?> GetEmailAsync(User user, CancellationToken cancellationToken) =>
    Task.FromResult<string?>(user.Email);
    public Task<bool> GetEmailConfirmedAsync(User user, CancellationToken cancellationToken) =>
    Task.FromResult(true);
    public Task SetEmailConfirmedAsync(User user, bool confirmed, CancellationToken cancellationToken) =>
    Task.CompletedTask;
```

Hình 6. Kho người dùng tùy chỉnh ánh xạ mật khẩu và email vào bảng User

4. Xác thực mật khẩu và quản lý tài khoản

Đăng nhập dùng UserManager.CheckPasswordAsync để kiểm tra mật khẩu đã băm. Đổi mật khẩu dùng UserManager.ChangePasswordAsync, qua đó áp dụng bộ kiểm tra mật khẩu của Identity; nếu thành công, hệ thống ghi AuditLog. Mật khẩu gốc không được ghi vào cơ sở dữ liệu hoặc Cookie.

Services/AccountService.cs — SOURCE LINES 19-22

```
19            bool passwordValid;
20            try { passwordValid = await userManager.CheckPasswordAsync(user, m.Password); }
21            catch (FormatException) { return null; }
22            if (!passwordValid) return null;
```

Hình 7. Kiểm tra mật khẩu khi đăng nhập bằng UserManager trong AccountService.cs

Services/AccountService.cs — SOURCE LINES 44-55

```
44            IdentityResult result;
45            try { result = await userManager.ChangePasswordAsync(user, m.CurrentPassword, m.NewPassword); }
46            catch (FormatException) { return ServiceResult.Failure("Mật khẩu hiện tại không đúng."); }
47            if (!result.Succeeded)
48            {
49                if (result.Errors.Any(e => e.Code == "PasswordMismatch"))
50                    return ServiceResult.Failure("Mật khẩu hiện tại không đúng.");
51                return ServiceResult.Failure(string.Join(" ", result.Errors.Select(e => e.Description)));
52            }
53            db.AuditLogs.Add(ServiceSupport.Audit(userId, "ChangePassword", "User", userId));
54            await db.SaveChangesAsync(ct);
55            return ServiceResult.Success(userId);
```

Hình 8. Đổi mật khẩu qua Identity và ghi nhật ký thao tác

Sau khi kiểm tra mật khẩu, ứng dụng tạo cookie xác thực chứa UserId, tên, email và vai trò. RememberMe quyết định cookie đăng nhập có được duy trì qua lần đóng trình duyệt hay không.

5. Phân quyền theo vai trò và dữ liệu

Cookie xác thực có thời hạn tối đa tám giờ, đặt HttpOnly, SameSite=Lax và Secure khi dùng HTTPS. Mỗi yêu cầu được kiểm tra lại với cơ sở dữ liệu để từ chối tài khoản bị khóa, phòng ban bị khóa hoặc vai trò không còn khớp.

Program.cs — SOURCE LINES 54-72

```
54  builder.Services.AddAuthentication(CookieAuthenticationDefaults.AuthenticationScheme).AddCookie(options =>
55  {
56      options.LoginPath = "/Account/Login";
57      options.AccessDeniedPath = "/Account/AccessDenied";
58      options.ExpireTimeSpan = TimeSpan.FromHours(8);
59      options.Cookie.HttpOnly = true;
60      options.Cookie.SameSite = SameSiteMode.Lax;
61      options.Cookie.SecurePolicy = CookieSecurePolicy.SameAsRequest;
62      options.Events.OnValidatePrincipal = async context =>
63      {
64          var account = context.HttpContext.RequestServices.GetRequiredService<IAccountService>();
65          if (!int.TryParse(context.Principal?.FindFirstValue(ClaimTypes.NameIdentifier), out var userId)
66              || !await account.ValidateSessionAsync(userId, context.Principal?.FindFirstValue(ClaimTypes.Role) ??
    ""))
67          {
68              context.RejectPrincipal();
69              await context.HttpContext.SignOutAsync();
70          }
71      };
72  });
```

Hình 9. Cấu hình cookie xác thực và kiểm tra lại tài khoản trong Program.cs

Tạo, sửa và hủy cuộc họp yêu cầu vai trò Organizer hoặc Admin. Ngoài kiểm tra vai trò, người tổ chức chỉ được sửa/hủy cuộc họp do mình tạo; Admin có thể quản lý toàn hệ thống.

Controllers/MeetingController.cs — SOURCE LINES 27-31

```
27      [Authorize(Roles = "Organizer,Admin"), HttpGet]
28      public async Task<IActionResult> Create(CancellationToken ct)
29      {
30          await PopulateLookupsAsync(ct);
31          return View(new MeetingEditViewModel());
```

Hình 10. Giới hạn chức năng tạo lịch theo vai trò trong MeetingController.cs

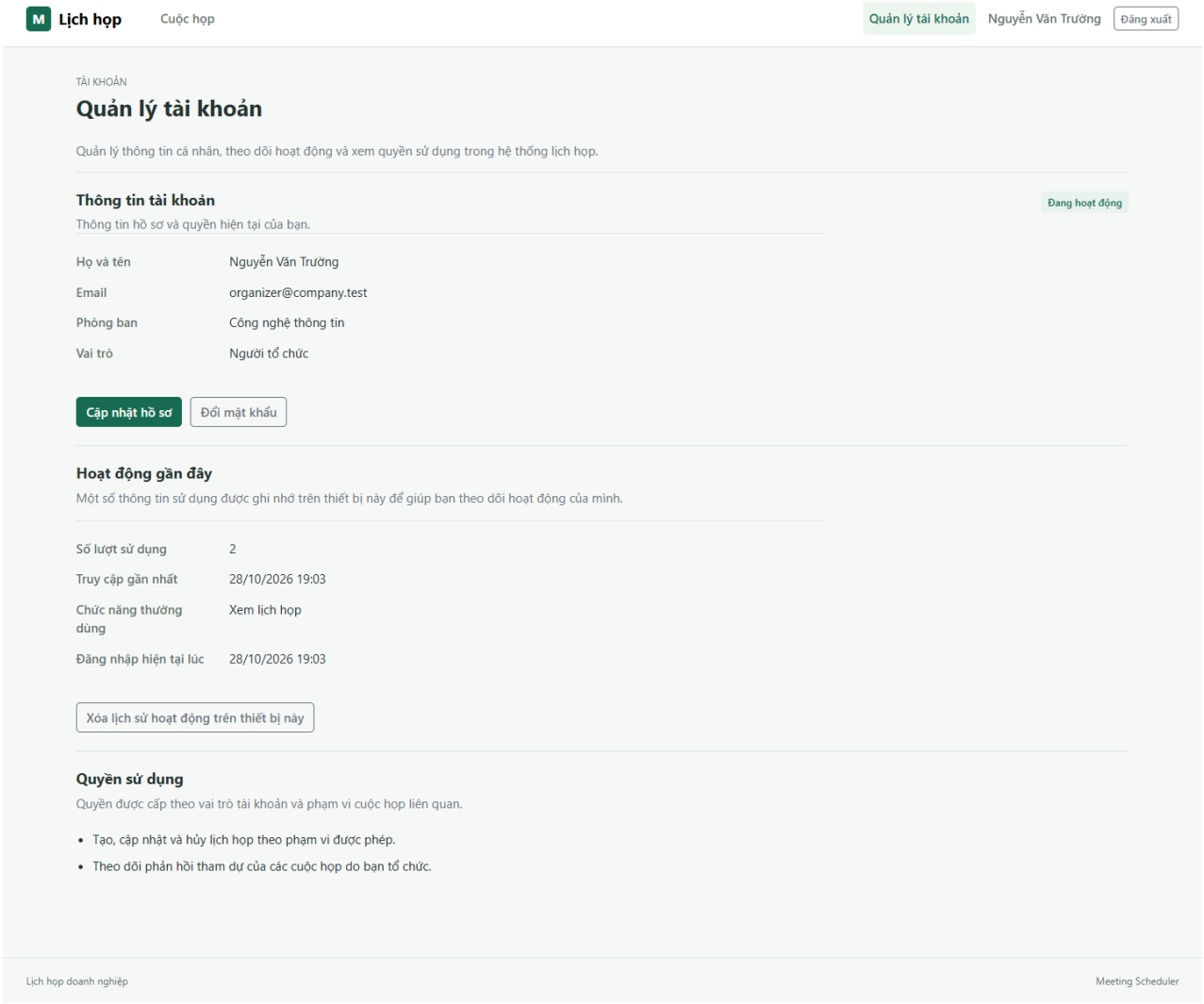
Controllers/MeetingController.cs — SOURCE LINES 74-79

```
74      public async Task<IActionResult> Cancel(int id, CancellationToken ct)
75      {
76          var meeting = await meetings.GetAsync(id, CurrentUserId, ct);
77          if (meeting is null) return NotFound();
78          if (!User.IsInRole("Admin") && meeting.CreatedByUserId != CurrentUserId) return Forbid();
79          ViewBag.Meeting = meeting;
```

Hình 11. Kiểm tra quyền sở hữu cuộc họp trước khi cho phép hủy lịch

6. Kết quả chạy ứng dụng

Ứng dụng chạy trên ASP.NET Core MVC và kết nối LocalDB đã cấu hình. Trang Quản lý tài khoản hiển thị thông tin hồ sơ, hoạt động gần đây và quyền sử dụng theo vai trò Organizer; không hiển thị chi tiết kỹ thuật nội bộ về cơ chế lưu trạng thái.



Hình 12. Trang Quản lý tài khoản sau khi đăng nhập với vai trò Người tổ chức

Cách chạy và kiểm tra

Từ thư mục MeetingScheduler, chạy lệnh: dotnet run --project .\MeetingScheduler.Web\MeetingScheduler.Web.csproj --urls http://localhost:5086. Đăng nhập bằng tài khoản demo organizer@company.test với mật khẩu Demo@12345, sau đó mở /Account/Security.

- Organizer được tạo, cập nhật, hủy lịch trong phạm vi được phép; không có quyền quản trị nhân sự và danh mục.
- Kiểm tra đăng xuất: Session bị xóa và cookie xác thực bị thu hồi.
- Build gần nhất: thành công, 0 cảnh báo và 0 lỗi. Đã mở trang tài khoản và xác nhận dữ liệu hiển thị đúng vai trò.

Kết luận

Các yêu cầu thực hành về Cookie, Session, Identity, xác thực và phân quyền đã được áp dụng vào quy trình quản lý lịch họp. Cấu trúc tài khoản và cơ sở dữ liệu hiện có được giữ nguyên, trong khi thao tác đăng nhập, đổi mật khẩu và giới hạn quyền được gắn trực tiếp với nghiệp vụ cuộc họp.

Tài liệu tham khảo

Lect7 ASP.NET Core MVC Security ver2.1, phần thực hành buổi 7 về Cookies, Session và Identity Framework.